

BLOGNEST API

AI-Powered Blog Content Management Backend

Technical Project Documentation

A secure and scalable REST API platform for blog creation, content management, role-based administration, and AI-assisted writing.

Project Overview

BlogNest is a modern backend platform designed to simplify blog creation, content management, and AI-assisted writing through a structured REST API. The system is implemented with Node.js and Express.js and uses MongoDB with Mongoose for data storage, schema definition, validation, and database operations.

The platform integrates Google Gemini AI to support intelligent features such as automated blog generation, frequently asked question assistance, WeatherWise insights, FitTrack recommendations, and content summarization. These services are designed to reduce manual effort while keeping generated content relevant to the user's context.

Security is incorporated throughout the backend architecture. JWT-based authentication provides stateless user authentication, while role-based access control separates permissions for Admin, Editor, Author, and Reader accounts. Passwords are protected using bcrypt.js, and additional controls include request validation, input sanitization, rate limiting, structured logging, and centralized error handling.

The result is a modular content-management backend that combines conventional blogging workflows with generative AI capabilities to provide a faster, more organized, and more personalized content-creation experience.

Scenario-Based Case Study

Background

Priya is a content creator who manages several blogs. As the number of posts increases, she finds it difficult to organize drafts, maintain categories and tags, schedule publications, moderate comments, and monitor reader engagement from a single workflow.

Challenges

- Managing multiple drafts, categories, and tags becomes difficult as the content library grows.
- Limited visibility into post performance makes it harder to evaluate reader engagement.
- Content approval and comment moderation require repetitive manual work.
- Media files and their associated metadata are not handled through a unified process.
- Different contributors require controlled permissions for authoring, editing, and administration.

Solution

BlogNest addresses these challenges through a centralized and secure API layer that separates business logic from frontend delivery. A permission matrix controls access to services for content scheduling, category management, media registration, and user-role administration.

Usage

Author: Creates blog posts, manages media references, organizes drafts, and reviews performance data for their own content.

Editor/Admin: Uses protected endpoints to review content queues, update publication status, moderate comments, and manage user privileges.

Reader: Accesses published posts through public routes and can submit comments through the appropriate endpoints.

Outcome

By moving workflow controls into a dedicated backend, BlogNest reduces delays in content execution and improves consistency. Schema validation protects data integrity, authorization rules restrict administrative operations to permitted roles, and persistent records provide a foundation for tracking content performance and optimization metrics.

Software Requirements

- Operating System: Windows 10/11, macOS, or Linux.
- Node.js v16 or later: Runtime environment for the server and API logic.
- npm v8 or later: Package manager for project dependencies.
- Express.js: Web framework used to build REST API routes and middleware.
- MongoDB: NoSQL database for users, posts, categories, comments, and analytics data.
- Postman: API testing tool for validating requests and responses.
- Code Editor: Visual Studio Code or another suitable development environment.

Hardware Requirements

- Processor: Intel Core i5 8th Generation or later, AMD Ryzen 5, or equivalent.
- Memory: 8 GB minimum; 16 GB recommended when running MongoDB and multiple development processes.
- Storage: At least 1 GB of available workspace for the project and its dependencies.

Project Architecture

BlogNest follows a layered server architecture in which incoming API requests pass through validation, authentication, authorization, business-logic, and data-access layers before a structured response is returned.

Architecture Components

API Interface: Tools such as Postman send requests to defined API routes and receive structured JSON responses.

Express Server Gateway: Receives network requests, maps HTTP methods such as GET, POST, PUT, and DELETE, parses JSON payloads, and manages CORS headers.

Authentication and Role Middleware: Reads authentication tokens, validates the user's identity, and checks whether the user's role has permission to access a protected resource.

Service and Logic Modules: Apply business rules for blog states such as Draft, Pending Approval, Scheduled, and Published, as well as comment workflows and reporting.

Database Interface: Mongoose connects application logic with MongoDB and provides schema-based data operations.

AI Integration Layer: Connects the application to external AI services for automated blog generation and summarization through dedicated endpoints.

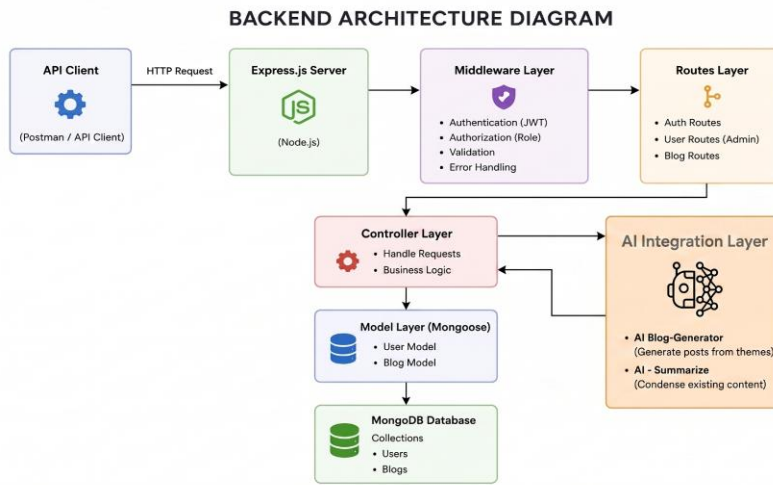


Figure 1. Enhanced project architecture / API flow.

AI and Blog Generation Service

The BlogGenerator service automates the initial drafting of blog content. It communicates with the AI integration layer so that a topic or high-level instruction can be transformed into a structured draft. The same service-oriented approach can be used for content summarization and other intelligent assistance features.

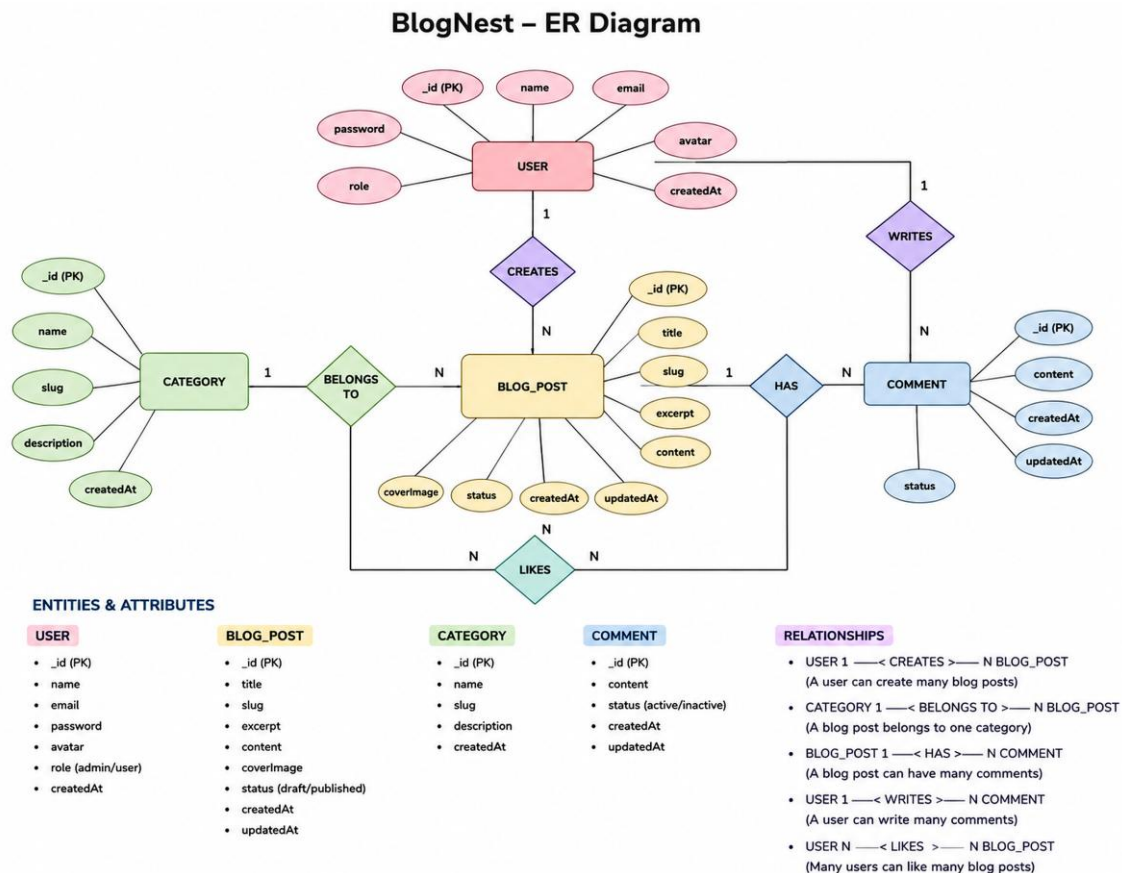


Figure 2. Enhanced service/entity representation.

Entity-Relationship Model

The database model is organized around logical relationships between users, posts, and comments. MongoDB stores the records as documents, while Mongoose schemas define the expected fields, data types, validation rules, and default values.

User Entity

- `_id` — Primary key using MongoDB ObjectId.
- `name` — Required user name.
- `email` — Required and unique email address.
- `password` — Required hashed password.
- `role` — User role with a default of Reader.

Post Entity

- `_id` — Primary key using MongoDB ObjectId.
- `userID` — Reference to the associated User document.
- `name` — Required post title.
- `photo` — Image or media path reference.
- `message` — Main blog content.
- `likes` — Numeric engagement count with a default of zero.
- `timestamps` — Automatically records creation and update times.

Key Relationships

- User to Post: One user can create multiple blog posts.
- User to Comment: A user can submit multiple comments across different posts.
- Post to Comment: A post can contain multiple individual comment records.

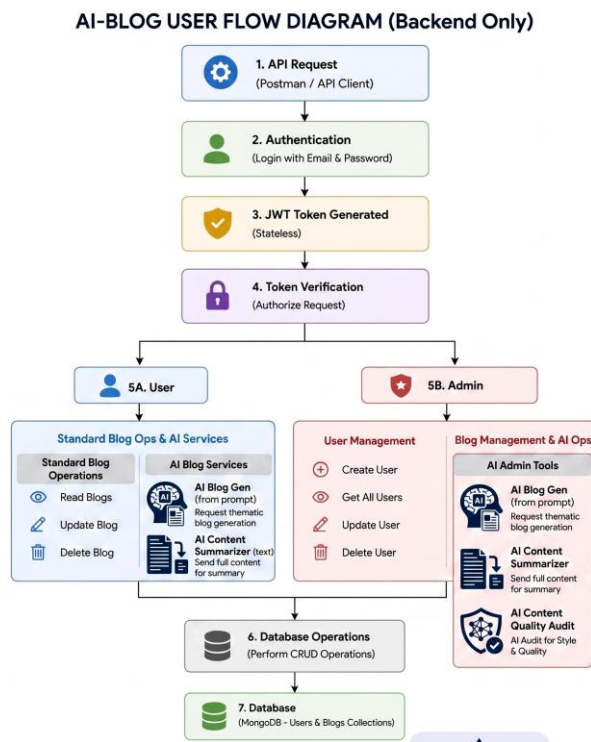


Figure 3. Enhanced database/entity visualization.

Key Features

- **Role-Based Route Protection:** Authentication and authorization checks prevent unauthorized users from modifying protected resources.
- **CRUD Blog Management:** Supports creating, reading, updating, and deleting blog content while maintaining defined status and asset information.
- **Categorization:** Categories and tags improve organization, filtering, and content discovery.
- **Comment Moderation:** Comments can be managed through protected endpoints with states such as approved, pending, or spam.
- **Input Sanitization:** Incoming parameters are sanitized to reduce risks from XSS and NoSQL injection-style attacks.
- **AI Blog Generation:** Large Language Model integration converts high-level topics and prompts into structured blog drafts.
- **Content Summarization:** Long articles can be processed to produce concise summaries and metadata.

Roles and Responsibilities

Admin: Has the broadest administrative permissions, including user management, role assignment, comment cleanup, and access to system logs.

Editor: Manages editorial workflows, reviews pending drafts, updates publication status, and maintains categories.

Author: Creates and manages posts associated with the author's account within the permissions granted by the system.

Reader: Views published content and submits comments using an authenticated account.

User Flow and MVC Architecture

The backend follows the Model-View-Controller pattern to separate data definitions, request processing, and API routing. Because BlogNest is a headless backend, the conventional user-interface view is replaced by an API routing layer. API routing layer.

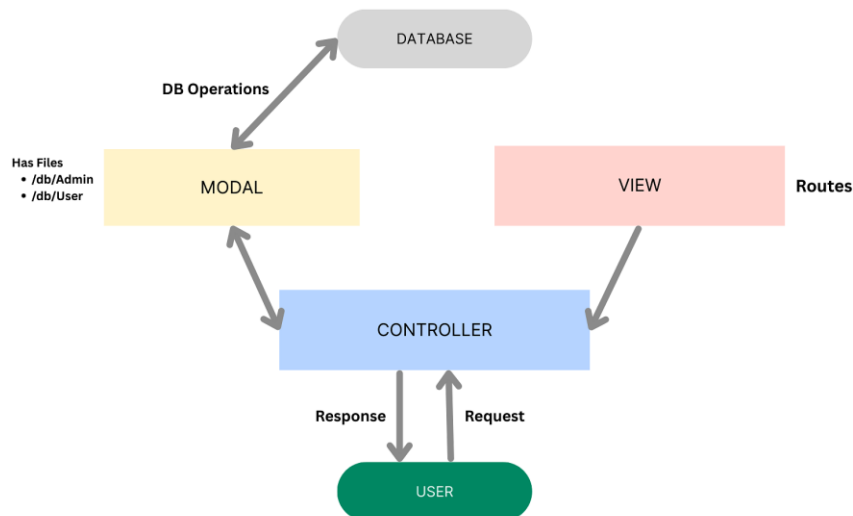


Figure 4. Enhanced user-flow / MVC representation.

Model Layer — Data Blueprint

The Model layer defines Mongoose schemas and maps application fields to MongoDB collections. It establishes validation rules, default values, indexing behavior, and transformation requirements before records are persisted.

Controller Layer — Execution Logic

The Controller layer receives requests from the routes, validates request requirements, coordinates business operations, communicates with the models, and returns standardized JSON responses.

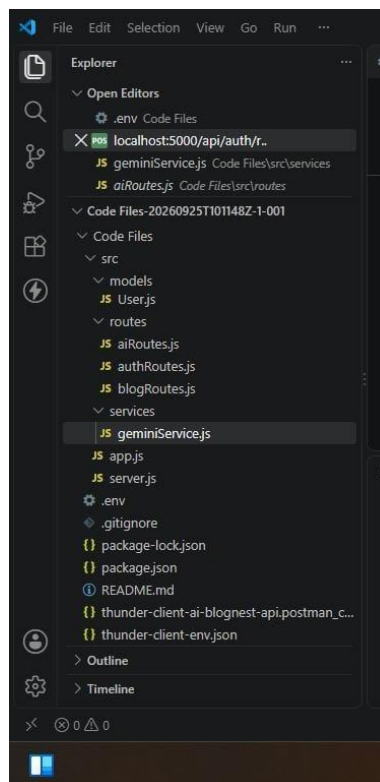
View Layer — API Routing Interface

BlogNest does not use traditional server-rendered pages. Instead, the API routing layer acts as the interface between HTTP requests and controller handlers, providing structured endpoints for clients.

Project Setup and Configuration

Creating the Project Folder

1. Create a new folder using the desired project name.
2. Create the required project subfolders inside the main directory.
3. Create a Server folder for the backend application.
4. Open the Server folder in Visual Studio Code.



Server Initialization

Open the Server folder in the VS Code terminal and initialize the Node.js project:

```
npm init -y
```

Create the main server file:

server.js

Create the following directories:

- models
- controllers
- routes

```
JS server.js X
Code Files > src > JS server.js
1  const dotenv = require('dotenv');
2  const app = require('./app');
3  const connectDB = require('./config/db');
4
5  dotenv.config();
6  connectDB();
7
8  const PORT = process.env.PORT || 5000;
9  app.listen(PORT, () => {
10   console.log(`Server running on port ${PORT}`);
11 });
12
```

Server.js.

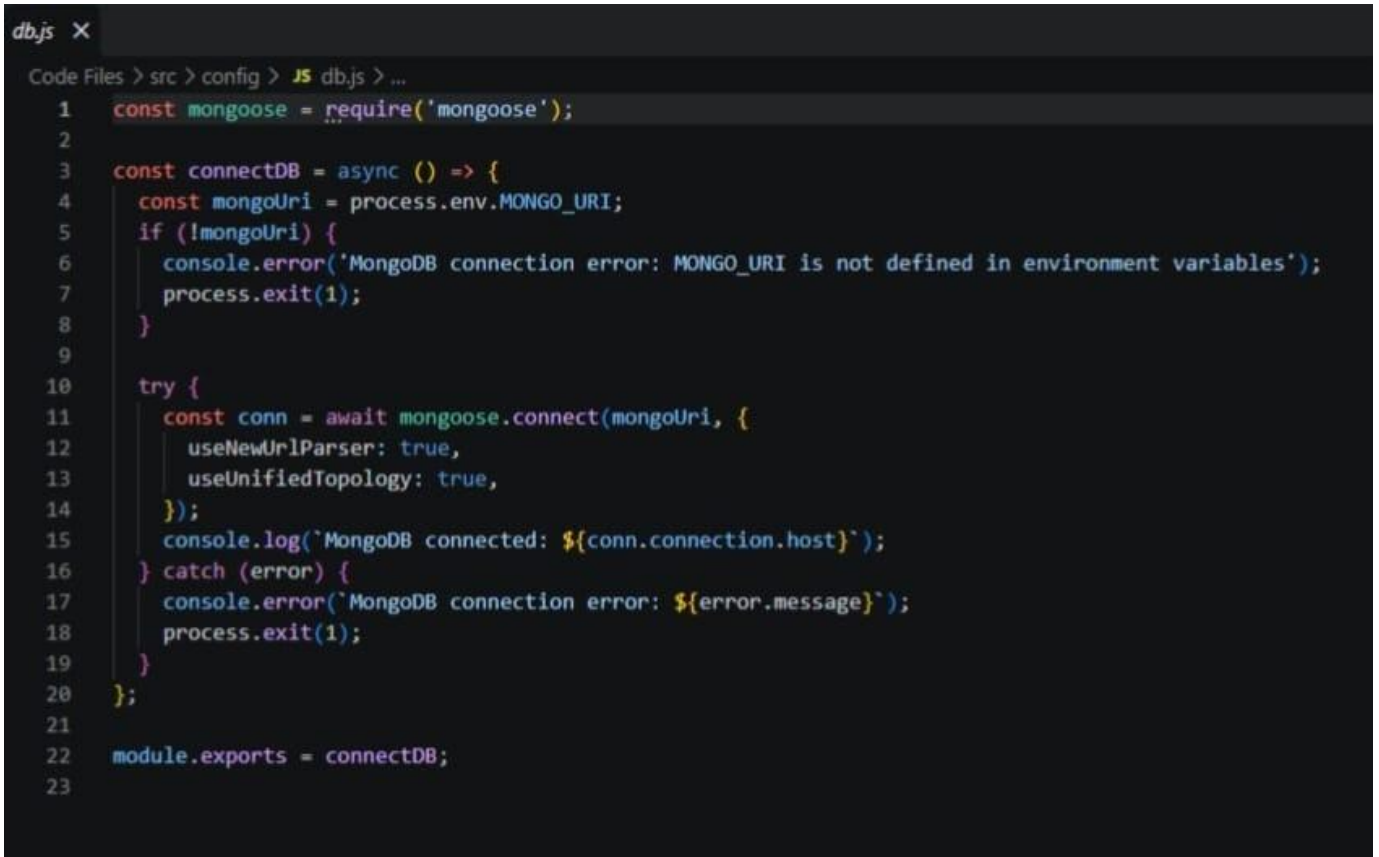
```
JS app.js X
src > JS app.js > ...
1  const express = require('express');
2  const cors = require('cors');
3  const morgan = require('morgan');
4  const authRoutes = require('./routes/authRoutes');
5  const blogRoutes = require('./routes/blogRoutes');
6  const aiRoutes = require('./routes/aiRoutes');
7  const errorMiddleware = require('./middleware/errorMiddleware');
8
9  const app = express();
10
11 app.use(cors());
12 app.use(express.json());
13 app.use(morgan('dev'));
14
15 app.use('/api/auth', authRoutes);
16 app.use('/api/blogs', blogRoutes);
17 app.use('/api/ai', aiRoutes);
18
19 app.use(errorMiddleware);
20
21 app.get('/', (req, res) => {
22   res.json({ message: 'AI BlogNest API is running' });
23 });
24
25 module.exports = app;
26
```

App.js

Backend Development

The backend implementation is centered around the Express server and the application configuration. The server initializes middleware, routes, database connectivity, and error handling, while application modules organize the API logic into maintainable components.

. Db connection

A screenshot of a code editor window titled 'db.js'. The editor shows a JavaScript file with the following code:

```
1  const mongoose = require('mongoose');
2
3  const connectDB = async () => {
4    const mongoUri = process.env.MONGO_URI;
5    if (!mongoUri) {
6      console.error('MongoDB connection error: MONGO_URI is not defined in environment variables');
7      process.exit(1);
8    }
9
10   try {
11     const conn = await mongoose.connect(mongoUri, {
12       useNewUrlParser: true,
13       useUnifiedTopology: true,
14     });
15     console.log(`MongoDB connected: ${conn.connection.host}`);
16   } catch (error) {
17     console.error(`MongoDB connection error: ${error.message}`);
18     process.exit(1);
19   }
20 };
21
22 module.exports = connectDB;
23
```

Database Configuration

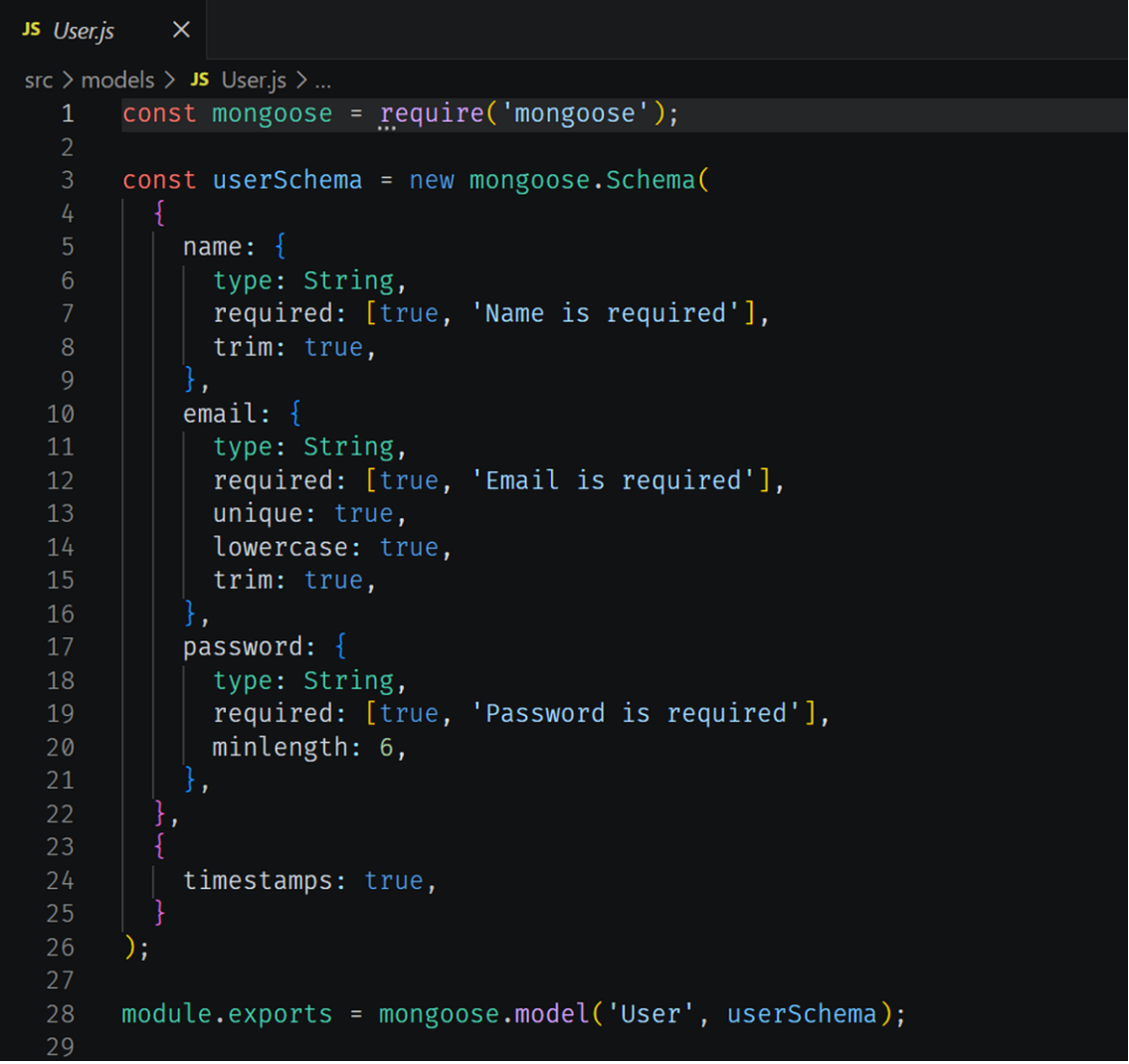
MongoDB Connection

MongoDB provides the document-oriented storage layer for BlogNest. Mongoose is used to define schemas, validate incoming data, and manage the connection between the Node.js application and MongoDB.

Install Mongoose from the Node.js project directory:

```
npm install mongoose
```

5. Configure the MongoDB connection string and establish the database connection.
6. Create the required Mongoose schemas and models.
7. Verify that the database connection is active before executing operations that require persistent data.



```
JS User.js  X
src > models > JS User.js > ...
1  const mongoose = require('mongoose');
2
3  const userSchema = new mongoose.Schema(
4    {
5      name: {
6        type: String,
7        required: [true, 'Name is required'],
8        trim: true,
9      },
10     email: {
11       type: String,
12       required: [true, 'Email is required'],
13       unique: true,
14       lowercase: true,
15       trim: true,
16     },
17     password: {
18       type: String,
19       required: [true, 'Password is required'],
20       minlength: 6,
21     },
22   },
23   {
24     timestamps: true,
25   }
26 );
27
28 module.exports = mongoose.model('User', userSchema);
29
```

Mongoose scheme definitions.

Identity Resource Document Schema

The user schema defines the validation requirements for identity records created through the backend. Required fields, unique values, role defaults, and password handling rules are applied before data is stored.

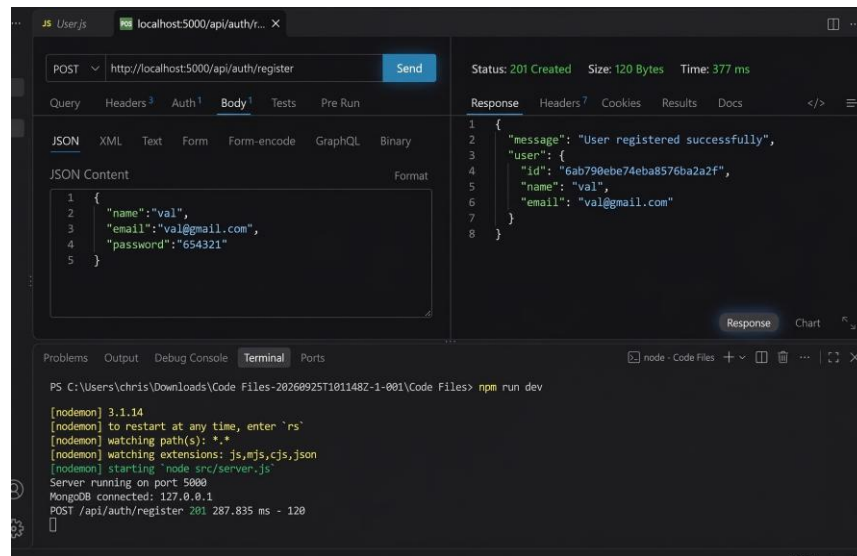
Project Execution and API Testing

For API verification, Postman or the Thunder Client extension for Visual Studio Code can be used. After the database and application configuration are complete, start the backend server with:

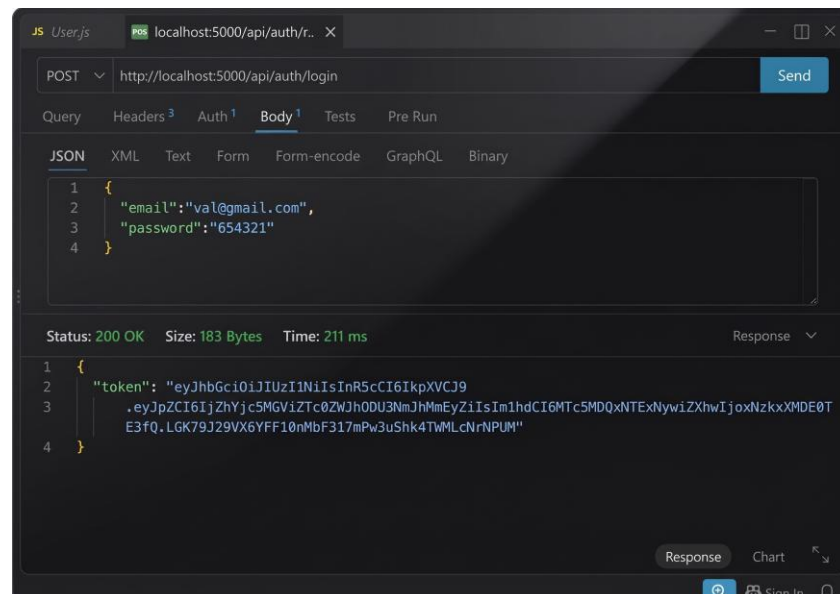
npm start

API Testing Workflow

The main API workflows include user registration, user authentication, blog creation, blog retrieval, AI-assisted blog generation, and content summarization. Each endpoint should be tested for valid requests, validation failures, authentication requirements, authorization behavior, and expected JSON responses.



user registration.



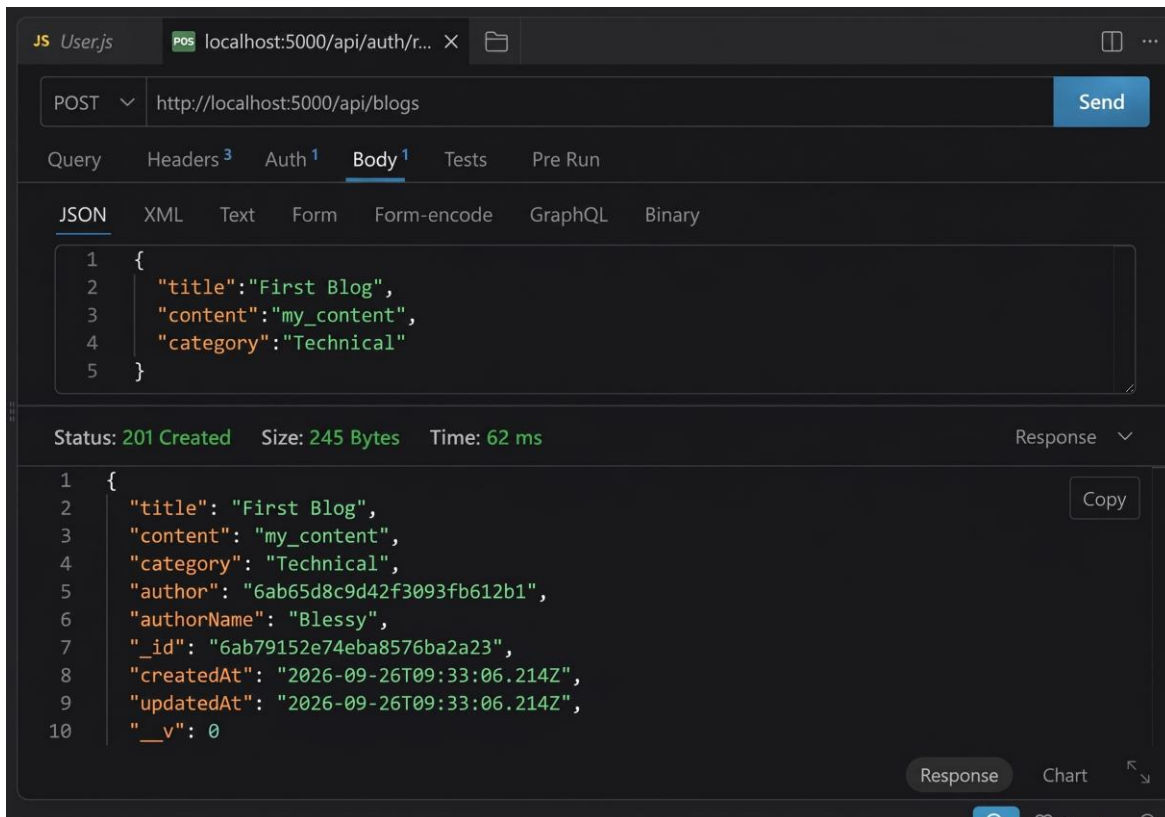
user login.

Blog Management APIs

Create Blogs

Authorized users can submit new blog posts through the blog creation endpoint. The request is validated before the post is stored, and the authenticated user's identity is associated with the created record.

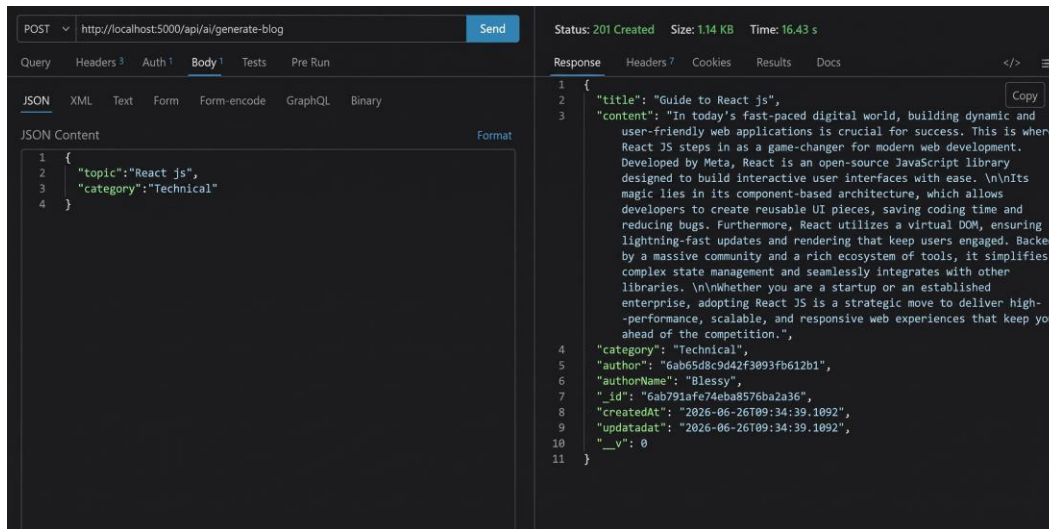
create blogs.



AI-Powered Content Services

Generate AI Blogs

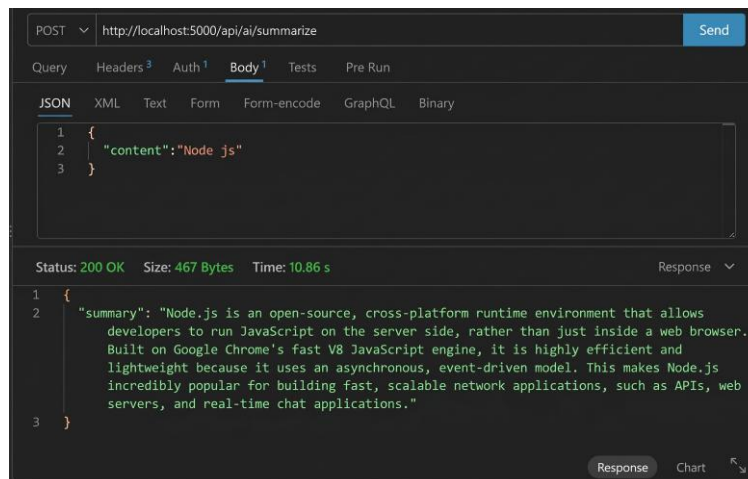
The AI blog-generation endpoint accepts a high-level topic or prompt and passes the request to the configured AI service. The returned result can be used as a structured starting point for a new blog post.



AI blog generation.

Summarize Blogs

The summarization endpoint processes existing blog content and returns a shorter representation that can be used for previews, metadata, or quick content review.



blog summarization.

Demonstration and Project Resources

The original documentation references demonstration videos for the project and API testing, along with the project's CodeDrive resources. These materials can be retained as supporting references for the final project demonstration and source-code review.

Referenced resources:

- [Demo Video](#) — Project demonstration
- [API Testing Video](#) — API testing demonstration
- [CodeDrive](#) — Project source-code resources

Conclusion

BlogNest provides a structured backend foundation for modern blog management by combining RESTful API design, MongoDB-based persistence, role-based authorization, secure authentication, and AI-assisted content services. Its layered architecture separates responsibilities across routes, controllers, services, models, and database operations, making the platform easier to maintain and extend.

The project demonstrates how conventional content-management workflows can be enhanced with generative AI while retaining controls for authentication, authorization, validation, and data integrity. The result is a flexible backend suitable for supporting a modern blogging application and future intelligent content-management features.